Chapter 2

Relational Data Model

Unit 2.3: Normalization: Normalization Concepts, Need of Normalization, Types of Normalization – 1NF, 2NF, 3NF

Prepared By

Mr. Mohammad Ali

Lecturer (Selection Grade)

M. H. Saboo Siddik Polytechnic, Mumbai

Learning Outcomes

The Learners will be able to:

1. Understand the concept of Normalization.
2. Design Normalized Database Structure in the given problem.

Normalization Concepts

- **Database Normalization** is the process of efficiently organizing data in a database. There are two goals of the normalization process:
  - **Eliminating redundant data** from the tables to improve
    - Storage efficiency,
    - Data Integrity and
    - Scalability.
  - **Ensuring data dependencies** (only storing related data in a table).
- **It divides larger tables to smaller(multiple) tables** and **link them using relationships.**
- The inventor of the relational model, **Dr. Edgar F. Codd** proposed the theory of normalization with the introduction of **First Normal Form, Second** and **Third Normal Form.**
- Later he joined with **Raymond F. Boyce** to develop the theory of **Boyce-Codd Normal Form (BCNF)**.

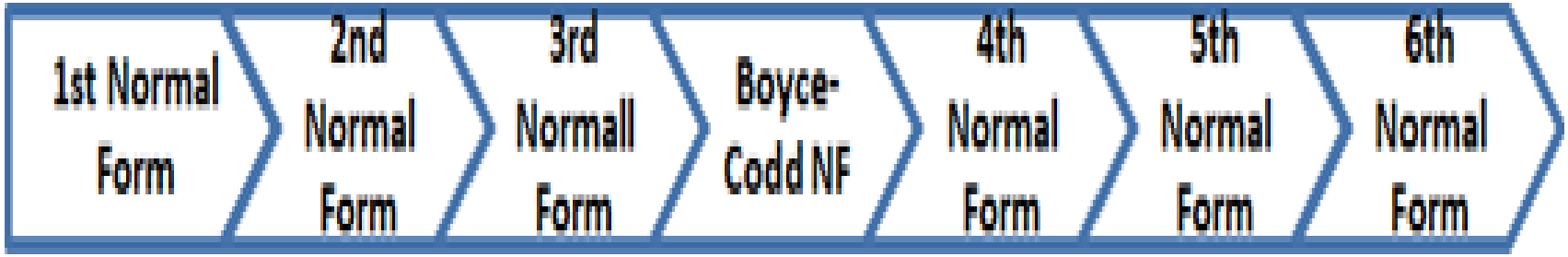
Need of Normalization

- Eliminate certain types of data (redundancy/replication) to improve consistency
- Enforce data integrity
- Produce a clearer and readable data model
- Provide maximum flexibility to meet future information needs by keeping tables corresponding to object types in their simplified forms.
- Avoid update anomalies.

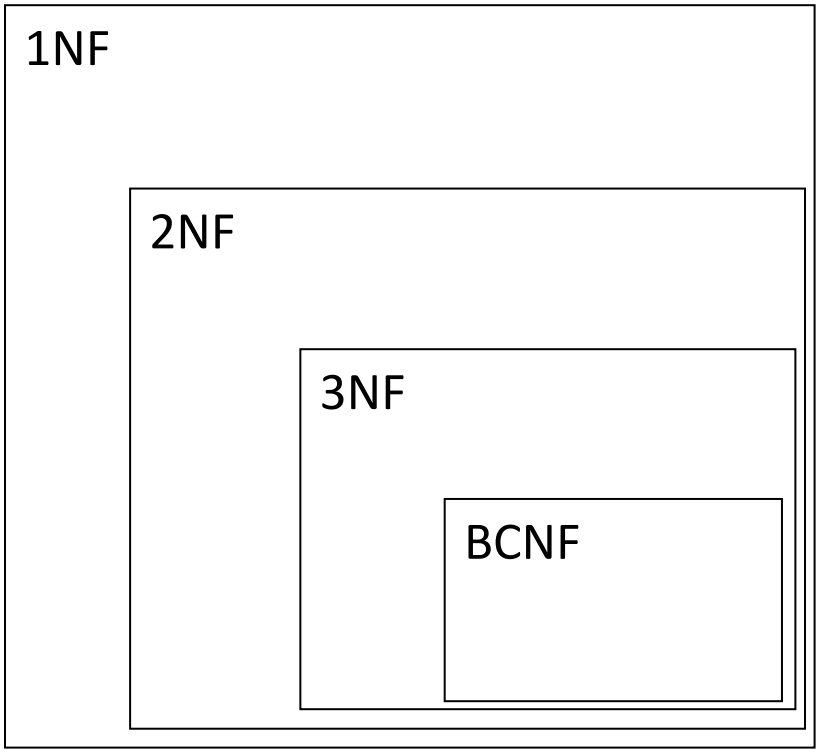
- **An update anomaly is a problem with:**
  - inserting (no place to insert new information),
  - deleting (loss of information),
  - or updating (inconsistency may occur because of the existence of data redundancy) a database because of the structure of the relations.

Types of Normalization

- There are a few rules for database normalization.
- Each rule is called a "N**ormal Form**."
- Dr. Edgar F. Codd originally established three normal forms: 1NF, 2NF and 3NF.
- If the first rule is observed, the database is said to be in **"First Normal Form (1NF)".**
- If the first three rules are observed, the database is considered to be in **"Third Normal Form (3NF)."**
- Although other levels of normalization are possible, third normal form is considered the highest level necessary for most applications.
- The evolution of Normalization theories is illustrated below-



Types of Normalization



a relation in BCNF, is also in 3NF

a relation in 3NF is also in 2NF

a relation in 2NF is also in 1NF

Functional Dependencies

- A functional dependency is an association between two attributes of the same relational database table.
- One of the attributes is called the **determinant** and the other attribute is called the **determined**.
- For each value of the **determinant** there is associated one and only one value of the **determined**.
- The left-hand side of the FD is called the **determinant**, and the right-hand side is the **dependent**.
- If **A** is the **determinant** and **B** is the **determined** then we say that ***A functionally determines B*** and graphically represent this as **A -> B**. The symbols A -> B can also be expressed as ***B is functionally determined by A***.
- We say that for a relation R has **Functional Dependency, A $\longrightarrow$ B** if the following conditions hold for every pairs of tuples t_1 and t_2 in R.

  If $t_1[A] = t_2[A]$ then $t_1[B] = t_2[B]$

  - For any two records, which have the same value for A, then the values for B in these two records must be the same.

Functional Dependencies

Let us consider the example:

- Let A = {Managerid, Managername} and
  B = {Managerage}
- We note that for the tuples t_1 and t_2 having same Managerid value and same Managername, we surely have same Managerage. Else, we know that data is inconsistent.
- Thus we can say that,

  {Managerid, Managername} ⟶ **{Managerage}**
- Functional dependency defines Boyce-Codd normal form and third normal form.
- This preserves dependency between attributes, eliminating the repetition of information.
- Functional dependency is related to a candidate key, which uniquely identifies a tuple and determines the value of all other attributes in the relation.

Functional Dependencies

- **Example** The following table illustrates A ⟶ B:

A	B
1	1
2	4
3	9
4	16
2	4
7	9

- Since for each value of A there is associated one and only one value of B.
- **Example** The following table illustrates that A does not functionally determine B:

A	B
1	1
2	4
3	9
4	16
3	10

- Since for A = 3 there is associated more than one value of B.

Functional Dependencies

Example:

- Suppose we keep track of employee email addresses, and we only track one email address for each employee.
- Suppose each employee is identified by their unique employee number.
- We say there is a functional dependency of email address on employee number:

  employee number → email address

EmpNum	EmpEmail	EmpFname	EmpLname
123	jdoe@abc.com	John	Doe
456	psmith@abc.com	Peter	Smith
555	alee1@abc.com	Alan	Lee
633	pdoe@abc.com	Peter	Doe
787	alee2@abc.com	Alan	Lee

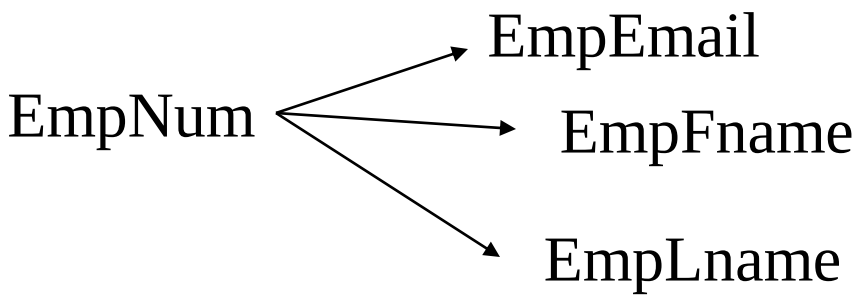
If EmpNum is the PK then the following FDs must exist:

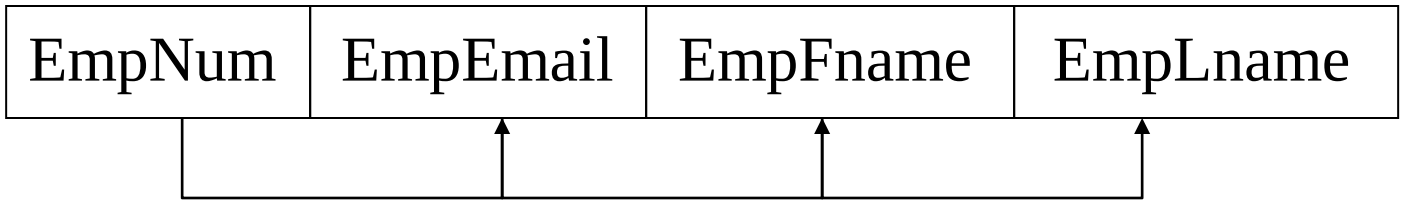
EmpNum → EmpEmail
EmpNum → EmpFname
EmpNum → EmpLname

Functional Dependencies

EmpNum → EmpEmail
EmpNum → EmpFname
EmpNum → EmpLname

3 different ways we might see FDs depicted





Transitive Dependencies

Transitive dependency

Consider attributes A, B, and C, and where

A → B and B → C.

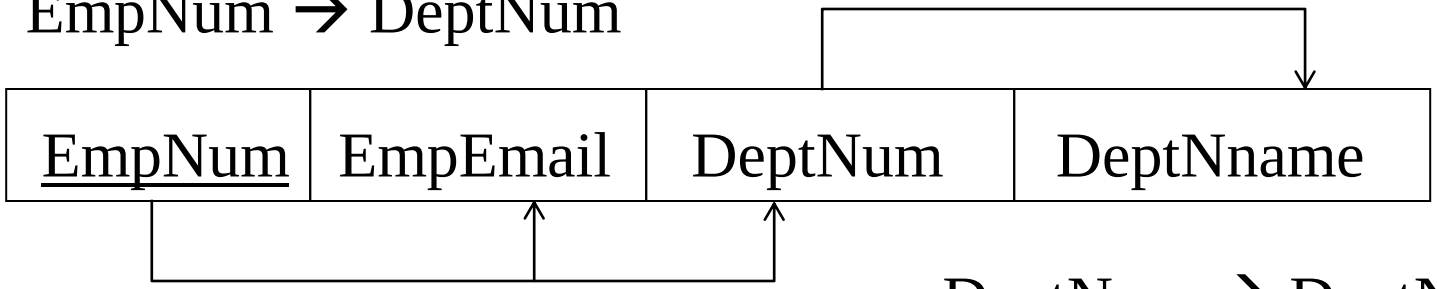
Functional dependencies are transitive, which means that we also have the functional dependency

A → C

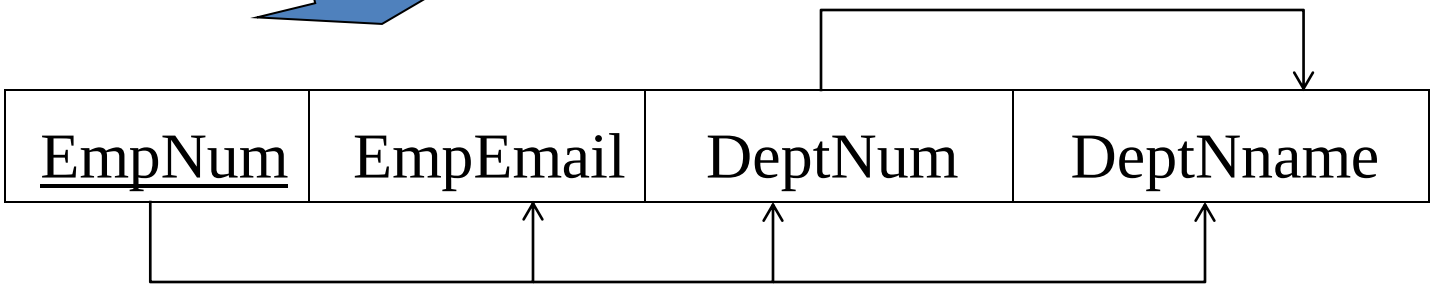
We say that C is transitively dependent on A through B.

Transitive Dependencies

EmpNum → DeptNum



DeptNum → DeptName

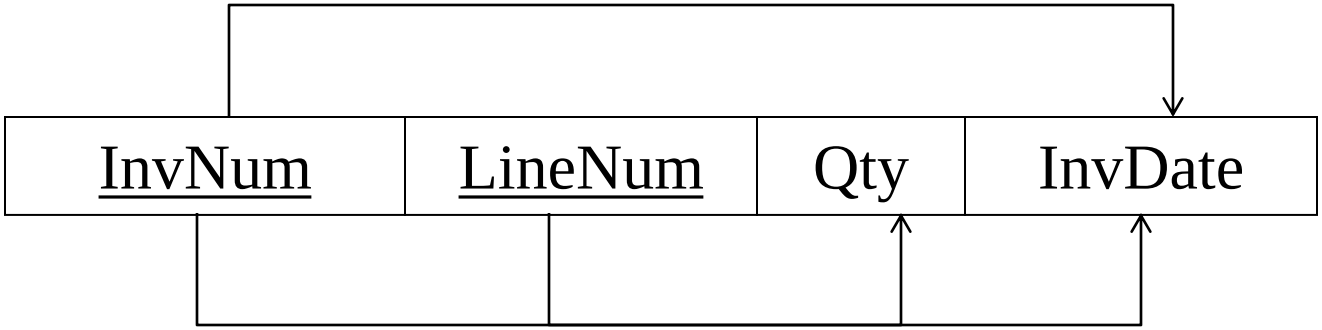


DeptName is *transitively dependent* on EmpNum via DeptNum

EmpNum → DeptName

Partial Dependencies

A **partial dependency** exists when an attribute B is functionally dependent on an attribute A, and A is a component of a multipart candidate key.



Candidate keys: {InvNum, LineNum} InvDate is *partially dependent* on {InvNum, LineNum} as InvNum is a determinant of InvDate and InvNum is part of a candidate key

Reflection Spot

We **decompose** a **relation** to **smaller relation** such that each **smaller relation** must be in required **Normal Form**.

How will you **decompose** a **relation** to **smaller relation**?

First Normal Form (1NF)

A relation schema is in First Normal Form (1NF) if the values in the relation instances are single-valued and atomic for every attribute in the relation.

For example:

In the relation **STUDENT**, let assume the attributes are STUDENT (RollNo, Name, DOB, Address)

Here, '**Address**' field may contain Flat no, Lane, City, State, Pin Code. Thus, 'address' field is non-atomic attribute. Hence, it is not in 1NF.

To put it in 1NF, we define a relation as follows:
ADDRESS (Ad_id, Flat_no, Lane, City, State, Pin)

And the **STUDENT** relation is redefined as:
STUDENT(Rollno, Name, DOB, Ad_id).

Hence, **ADDRESS** and **STUDENT** relations are in **1NF**.

First Normal Form (1NF)

The following relation(table) is **not** in **1NF**

Teacher_id	Name	Dept	Degrees
121	Mohammad Ali	Computer	B.E., M. Tech
122	Shafaque J	Computer	B.E., M. Tech
123	Mohammed Zaid	Computer	B.E., M. Tech

Degrees is a multi-valued attribute:

All the teachers have two degrees: *B.E.* and *M. Tech*

To obtain 1NF relations we must replace the above relation with two relations (without loss of information) – shown on next slide

First Normal Form (1NF)

The following relations(tables) are in **1NF**

Teacher

Teacher_id	Name	Dept
121	Mohammad Ali	Computer
122	Shafaque J	Computer
123	Mohammed Zaid	Computer

TeacherDegree

Teacher_id	Degrees
121	B.E.
121	M. Tech
122	B.E.
122	M. Tech
123	B.E.
123	M. Tech

An outer join between Teacher and TeacherDegree will produce the information we saw before

Second Normal Form (2NF)

A relation schema is in 2NF if it is in 1NF and no non-key attribute is functionally dependent on just part of a key.

In other words, a relation schema is in 2NF if every non-key attribute is fully functionally dependent on the key(candidate key).

Hence, a relation is obviously in 2NF if the key contains a single attribute.

In case the key is composite key then there may be a possibility that it may not be in 2NF.

A *key attribute* is any attribute that is part of a key; any attribute that is not a key attribute, is a *non-key attribute*.

A relation in 2NF will not have any partial dependencies

Let us consider following relation schema:

- **OWNS**(name, city, street, date, acc_no, balance)
- Here, **name** & **acc_no** together constitute the **primary key** of relation **OWNS**.
- We assume that more than one customer can own an account.

We can see that the FD :

- {name, acc_no} ⟶ {date} is a fully functional dependency. But
- {name, acc_no} ⟶ {balance} is not a fully functional dependency because
- {acc_no} ⟶ {balance} is a fully functional dependency

Second Normal Form (2NF)

- We decompose the relation to smaller relations such that each of the resulting relations is in 2NF.
- Thus we have three relations,
  - **CUST(name, City, street)**
  - **ACC(acc_no, balance)**
  - **OWNS(name, acc_no, date)**
- Hence, Second normal form (2NF) further addresses the concept of removing duplicative data: (2NF Rules)
  - Meet all the requirements of the first normal form – Be in 1NF.
  - Single column Primary Key
  - Remove subsets of data that apply to multiple rows of a table and place them in separate tables.
  - Create relationships between these new tables and their predecessors through the use of **foreign keys**.

Second Normal Form (2NF)

Consider this **Invoice** table (it's in 1NF)**:**

InvNum	LineNum	ProdNum	Qty	InvDate

InvNum, LineNum ⟶ ProdNum, Qty

There are two candidate keys.

InvNum ⟶ InvDate

Invoice is **not** in **2NF** since there is a partial dependency of InvDate on InvNum

Invoice is only in **1NF**

Second Normal Form (2NF)

Invoice

InvNum	LineNum	ProdNum	Qty	InvDate

The above relation has redundancies: the invoice date is repeated on each invoice line.

We can *improve* the database by decomposing the relation into two relations:

InvNum	LineNum	ProdNum	Qty

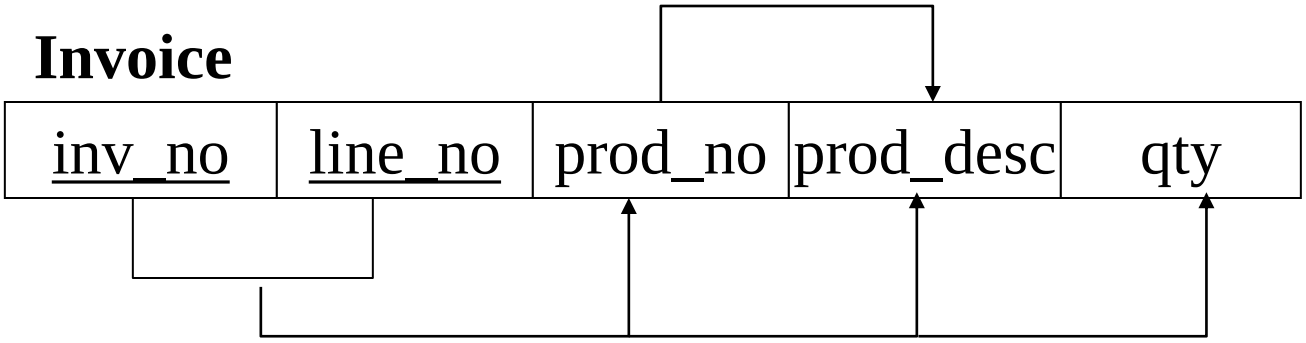
InvNum	InvDate

Second Normal Form (2NF)

Are the following relations in 2NF?

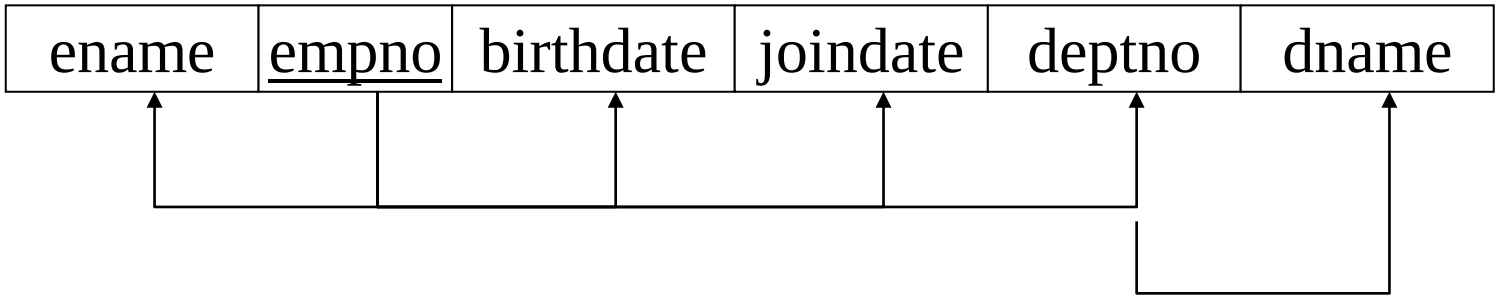
In 2NF, but not in 3NF:

Invoice



since prod_no is not a candidate key and we have:
prod_no → prod_desc

Employee



since deptno is not a candidate key and we have:
dnumber → dname

Third Normal Form (3NF)

- Third normal form (3NF) requires that there are no functional dependencies of non-key attributes on something other than a candidate key.
- A table is in 3NF if all of the non-primary key attributes are mutually independent
- In other words, A relation in 3NF will not have any transitive dependencies of non-key attribute on a candidate key through another non-key attribute.
- Third Normal Form is based on the concept of transitive dependency.
- A FD X? ⟶ Y in relation schema **R** is transitive dependency if there is a set of attributes Z that is not a subset of any key of **R** and both X⟶ Z and Z⟶Y hold
- 3NF Rules
  - Rule 1- Be in 2NF
  - Rule 2- Has no transitive functional dependencies
  - Rule 3- Remove columns that are not dependent upon the primary key.

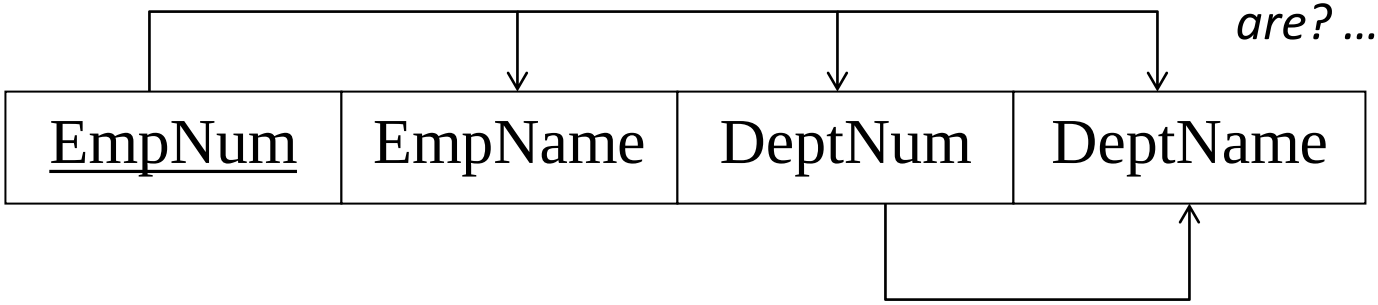
Third Normal Form (3NF)

- **A relation R is said to be in 3NF if it is in 2NF and no non-key attribute of R is transitively dependent on the primary key.**
- Let us consider an example of the relation STUDENT having schema (name, rollno, dob, street, city, pincode)
- We have the following FDs for this realtion:
  - {rollno} $\longrightarrow$ {city}
  - {city} $\longrightarrow$ {pincode}
- Here, pincode is transitively dependent on the key rollno. And pincode is not a key attribute.
- Thus this relation is not in 3NF.
- If a relation is not in 3NF, the relation can be decomposed into smaller relations so that no relation contains any transitive dependency involving key attributes.
- Thus STUDEN relation can be decomposed as:
  - STUDENT (name, rollno, dob, street, city)
  - PIN(city, pincode)

Third Normal Form (3NF)

Consider this **Employee** relation

Candidate keys are? …



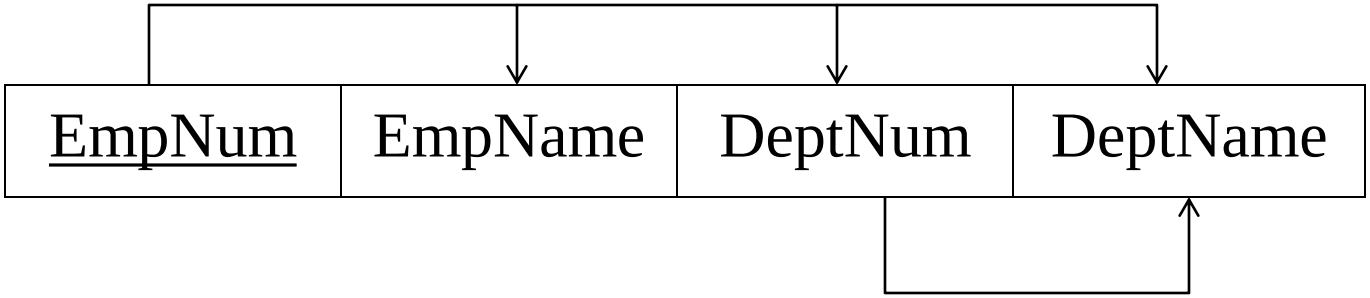
EmpName, DeptNum, and DeptName are non-key attributes.

DeptNum determines DeptName, a non-key attribute, and DeptNum is not a candidate key.

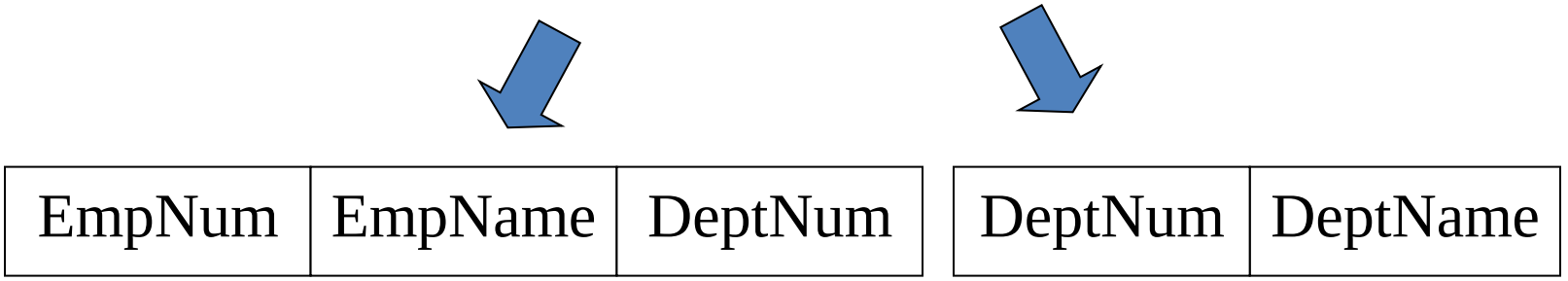
Is the relation in 3NF? **NO**

Is the relation in 2NF? **YES**

Third Normal Form (3NF)



We decompose the original relation into two 3NF relations. Note the decomposition is *lossless*.



These two relations are in 3NF.


Thanks
Finally, We normalized a relation.